

Below are **5 practice problems** covering **Simple (Linear) Queue**, **Circular Queue**, and **Priority Queue**, with `MAX = 5`. Each problem includes enough operations to encounter **overflow and/or underflow**.

Problem 1 – Simple Linear Queue

Assume `MAX = 5`.

Initially:

```
FRONT = -1  
REAR  = -1
```

Perform the following operations:

```
ENQUEUE (10)  
ENQUEUE (20)  
ENQUEUE (30)  
ENQUEUE (40)  
ENQUEUE (50)  
ENQUEUE (60)  
DEQUEUE ()  
DEQUEUE ()  
DEQUEUE ()  
DEQUEUE ()  
DEQUEUE ()  
DEQUEUE ()
```

Find after every operation:

1. Queue contents
 2. `FRONT`
 3. `REAR`
 4. Whether **OVERFLOW** or **UNDERFLOW** occurs
-

Problem 2 – Simple Linear Queue

Assume `MAX = 5`.

Initially:

```
FRONT = -1  
REAR  = -1
```

Perform:

```
DEQUEUE ()
ENQUEUE (11)
ENQUEUE (22)
ENQUEUE (33)
DEQUEUE ()
ENQUEUE (44)
ENQUEUE (55)
ENQUEUE (66)
ENQUEUE (77)
DEQUEUE ()
DEQUEUE ()
DEQUEUE ()
DEQUEUE ()
DEQUEUE ()
```

Determine the value of **FRONT** and **REAR** after each operation and identify all **overflow/underflow** conditions.

Problem 3 – Circular Queue

Assume:

```
MAX = 5
```

Initially:

```
FRONT = -1
REAR  = -1
```

Perform:

```
ENQUEUE (10)
ENQUEUE (20)
ENQUEUE (30)
ENQUEUE (40)
DEQUEUE ()
DEQUEUE ()
ENQUEUE (50)
ENQUEUE (60)
ENQUEUE (70)
ENQUEUE (80)
ENQUEUE (90)
DEQUEUE ()
DEQUEUE ()
DEQUEUE ()
DEQUEUE ()
```

```
DEQUEUE ()
DEQUEUE ()
```

For every operation, calculate:

- Queue array
- FRONT
- REAR
- Logical queue contents
- **Overflow/Underflow**, if any

Pay particular attention to the **wrap-around of REAR**.

Problem 4 – Circular Queue

Assume:

```
MAX = 5
```

Initially:

```
FRONT = -1
REAR  = -1
```

Perform:

```
ENQUEUE (5)
ENQUEUE (10)
ENQUEUE (15)
DEQUEUE ()
ENQUEUE (20)
ENQUEUE (25)
ENQUEUE (30)
DEQUEUE ()
ENQUEUE (35)
ENQUEUE (40)
ENQUEUE (45)
DEQUEUE ()
DEQUEUE ()
DEQUEUE ()
DEQUEUE ()
DEQUEUE ()
ENQUEUE (50)
ENQUEUE (55)
```

Determine after each operation:

```
Queue = ?  
FRONT = ?  
REAR = ?
```

Also identify every occurrence of:

```
OVERFLOW  
UNDERFLOW
```

Problem 5 – Priority Queue

Assume:

```
MAX = 5
```

Smaller priority number = higher priority.

For example:

```
Priority 1 = Highest  
Priority 2  
Priority 3  
Priority 4 = Lowest
```

Initially:

```
FRONT = -1  
REAR = -1
```

Perform:

```
DEQUEUE ()  
  
ENQUEUE (A, 3)  
ENQUEUE (B, 1)  
ENQUEUE (C, 4)  
ENQUEUE (D, 2)  
ENQUEUE (E, 1)  
  
ENQUEUE (F, 3)  
  
DEQUEUE ()  
DEQUEUE ()  
DEQUEUE ()  
DEQUEUE ()
```

DEQUEUE ()

DEQUEUE ()

For every operation, determine:

1. Queue contents
2. FRONT
3. REAR
4. Element deleted
5. **OVERFLOW/UNDERFLOW**, if applicable

Also remember that elements having the **same priority are processed in FIFO order**.

Student Task

For all 5 problems, prepare a table in this format:

Step	Operation	Queue	FRONT	REAR	Status
1	ENQUEUE(...)	—	—	—	—
2	ENQUEUE(...)	—	—	—	—
3	DEQUEUE()	—	—	—	—

Do not skip the overflow and underflow operations. Identify exactly **which operation causes OVERFLOW/UNDERFLOW** and why.